

Sure. Here is a **100-mark SQL examination paper** based on just **two sample tables**, with **3–4 rows of data** in each table. The questions progress from basic to advanced and cover the important SQL topics.

 SQL Examination — 100 Marks

Time: 2 Hours

Total Marks: 100

 **Table 1: Employees**

EmployeeID	EmployeeName	Department	Salary	City
101	Rahul	IT	60000	Kolkata
102	Priya	HR	45000	Delhi
103	Amit	IT	75000	Mumbai
104	Sneha	Sales	55000	Kolkata

 **Table 2: Departments**

DepartmentID	Department	Manager	Location
1	IT	Arjun	Kolkata
2	HR	Neha	Delhi
3	Sales	Rohan	Mumbai
4	Finance	Karan	Pune

Note: Assume `Employees.Department` and `Departments.Department` contain matching department names.

Section A — Basic SQL

20 Marks

Q1. Answer the following. (2 × 5 = 10)

- a) Display all records from the `Employees` table.
- b) Display only `EmployeeName`, `Department`, and `Salary`.
- c) Display employees whose salary is greater than 50,000.
- d) Display employees who belong to the IT department.
- e) Display employees who live in Kolkata.

Q2. (2 Marks)

Display all employees in descending order of salary.

Q3. (2 Marks)

Display the employees whose salary is between 50,000 and 70,000.

Q4. (2 Marks)

Display employees whose name starts with the letter 'A'.

Q5. (4 Marks)

Display the **highest salary**, **lowest salary**, **average salary**, and **total salary** of all employees.

Section B — Filtering, Functions & CASE

15 Marks

Q6. (3 Marks)

Display each employee's name in uppercase.

Q7. (3 Marks)

Display employees whose name contains the letter 'a'.

Q8. (4 Marks)

Create a new column called `Salary_Category` using `CASE` :

- Salary $\geq$ 70,000 → `High`
- Salary between 50,000 and 69,999 → `Medium`
- Salary $<$ 50,000 → `Low`

Q9. (5 Marks)

Display employee name, salary, and a calculated salary after giving everyone a **10%**

increment.

Section C — GROUP BY & Aggregate Functions

15 Marks

Q10. (5 Marks)

Find the **number of employees** in each department.

Q11. (5 Marks)

Find the **average salary** of each department.

Q12. (5 Marks)

Display only those departments whose **average salary is greater than 50,000**.

Section D — JOINS

20 Marks

Q13. (5 Marks)

Using an `INNER JOIN`, display:

- Employee Name
- Department
- Salary
- Manager

Q14. (5 Marks)

Using a `LEFT JOIN`, display **all employees along with their department manager**.

Q15. (5 Marks)

Display all departments and their employees, including departments that **do not have any employees**.

Q16. (5 Marks)

Find the departments that currently have **no employees**.

Section E — Subqueries

10 Marks

Q17. (5 Marks)

Find employees whose salary is **greater than the average salary of all employees**.

Q18. (5 Marks)

Find the employee(s) who receive the **highest salary** using a subquery.

Section F — Advanced SQL

20 Marks

Q19. (5 Marks)

Use a **CTE (Common Table Expression)** to calculate the average salary of each department and display the result.

Q20. (5 Marks)

Using a **window function**, assign a salary rank to every employee, where the highest salary gets Rank 1.

Expected concept:

```
RANK() / DENSE_RANK()
```

Q21. (5 Marks)

Using a window function, calculate the **difference between each employee's salary and the overall average salary**.

Q22. (5 Marks)

Using a window function, find the **highest-paid employee in each department**.

Section G — Practical Challenge

10 Marks

Q23. (10 Marks)

Write a single SQL query that displays:

Employee eName	Departme nt	Salary	Manager	Salary_R ank	Salary_C ategory
-------------------	----------------	--------	---------	-----------------	---------------------

The query must:

1. Join the two tables.
2. Display the employee's manager.
3. Rank employees according to salary.

4. Categorize salary as **High / Medium / Low** using `CASE`.
 5. Sort the final result by salary from highest to lowest.
-

Topic Coverage

SQL Topic	Covered
SELECT	✓
WHERE	✓
AND / BETWEEN	✓
LIKE	✓
ORDER BY	✓
Aggregate Functions	✓
GROUP BY	✓
HAVING	✓
CASE	✓
Calculations	✓
INNER JOIN	✓

LEFT JOIN	✓
NULL / Missing Data	✓
Subquery	✓
CTE	✓
Window Functions	✓
RANK / DENSE_ RANK	✓
Multi-table Query	✓

Total = 100 Marks.